

Agent Arena

Research Platform for Reinforcement Learning and Multi-Agent Terminal Agents

Status: Research / Early Development

Project Type: Open-source research platform, benchmark, and training environment

Primary Domains: Reinforcement Learning, Multi-Agent Systems, LLM Agents, Terminal Agents, Software Engineering, Cybersecurity, Systems Research

1. Abstract

Agent Arena is a research platform for studying autonomous AI agents operating inside realistic terminal-based environments.

The central research question is:

Can reinforcement learning learn effective orchestration policies for teams of autonomous terminal agents operating in complex, partially observable engineering environments?

Unlike conventional coding benchmarks that primarily evaluate whether an agent produces a correct answer or patch, Agent Arena evaluates the complete process by which an agent investigates an environment, forms hypotheses, executes actions, communicates with other agents, recovers from failures, and ultimately reaches a verified solution.

The platform combines four components:

1. **Realistic interactive environments**
2. **Multi-agent collaboration**
3. **Reinforcement-learning-based orchestration**
4. **Fine-grained trajectory evaluation**

The initial environment is a cryptographic challenge involving ECDSA nonce leakage and lattice-based private-key recovery. The existing challenge serves as a proof of concept for the broader framework.

Agent Arena is intended to evolve from a collection of difficult terminal tasks into a reproducible experimental platform for studying autonomous agent behavior.

2. Motivation

Modern language models are increasingly capable of operating computers through terminals, writing code, executing programs, inspecting files, debugging systems, and interacting with development environments.

However, traditional benchmarks frequently reduce performance to a binary outcome:

PASS / FAIL

This hides important properties of agent behavior.

Two agents may both solve a task while behaving very differently:

```
Agent A  
  
Inspect environment  
  ↓  
Form hypothesis  
  ↓  
Run targeted experiment  
  ↓  
Reject hypothesis  
  ↓  
Refine approach  
  ↓  
Solve
```

versus:

```
Agent B  
  
Random exploration  
  ↓  
Repeated failed commands  
  ↓  
Unnecessary modifications  
  ↓  
More exploration  
  ↓  
Eventually succeeds
```

Both receive a score of 1 under a binary benchmark.

They are not equally capable.

Agent Arena therefore treats the **trajectory itself as a research object**.

3. Research Objectives

Agent Arena will investigate the following questions.

3.1 Single-Agent Capability

How effectively can autonomous agents:

- explore unfamiliar environments?
- identify relevant information?
- formulate hypotheses?
- execute multi-step plans?
- recover from errors?
- operate under resource constraints?
- solve tasks without explicit procedural instructions?

3.2 Multi-Agent Collaboration

Does specialization improve performance?

For example:

Explorer
Researcher
Executor
Critic
Verifier

versus a single general-purpose agent.

3.3 Communication

How much communication is actually useful?

The system should investigate:

- when agents should communicate;
- what information should be communicated;
- how often communication should occur;

- whether excessive communication harms efficiency.

3.4 Learned Orchestration

Can a learned policy determine:

- which agent should act next;
- when to delegate;
- when to ask for review;
- when to retry;
- when to abandon an approach;
- when the task is sufficiently solved?

3.5 Reinforcement Learning

Can execution feedback be converted into useful reward signals for improving agent behavior?

3.6 Reliability

Can the framework measure not only task completion but also:

- recovery;
- efficiency;
- safety;
- unnecessary actions;
- destructive behavior;
- robustness to ambiguity;
- susceptibility to misleading information?

4. Core Hypothesis

The primary hypothesis is:

A learned orchestration policy over specialized agents can outperform both a single general-purpose agent and static multi-agent workflows on long-horizon terminal tasks, while reducing unnecessary computation and communication.

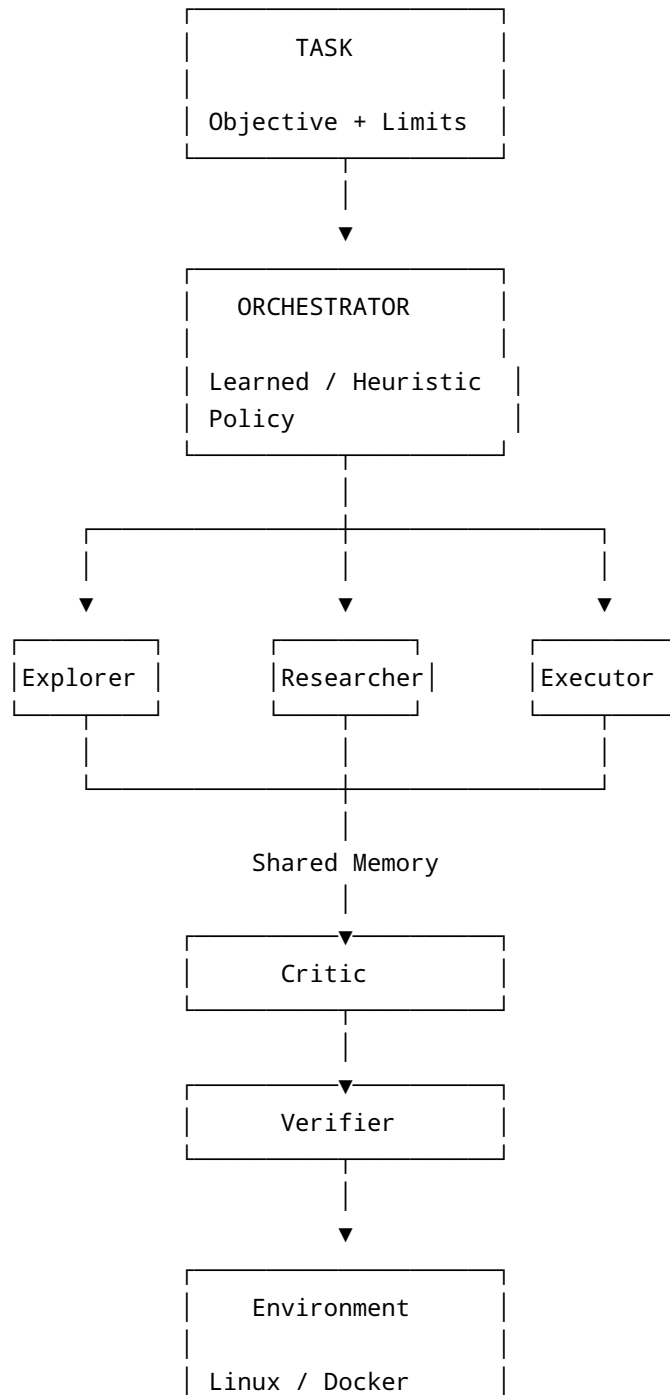
A secondary hypothesis is:

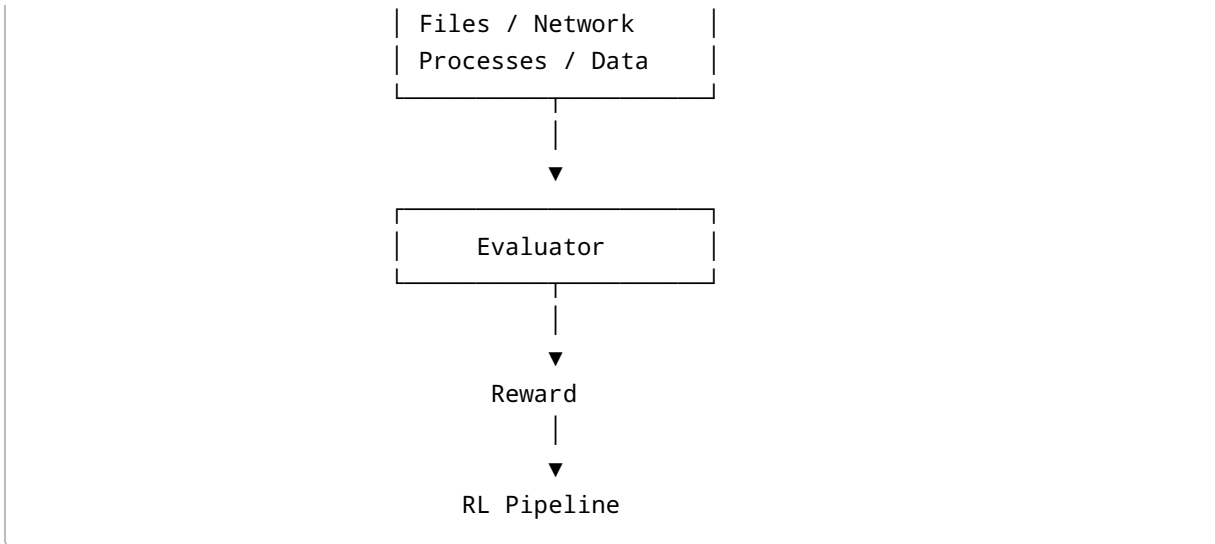
Fine-grained execution feedback provides a more informative training signal than binary task success alone.

A third hypothesis is:

The optimal number, specialization, and communication pattern of agents depends on task complexity and partial observability.

5. Conceptual Architecture





6. Environment Model

An Agent Arena environment is an interactive stateful system.

The environment may contain:

- source code;
- configuration files;
- databases;
- logs;
- binaries;
- network services;
- processes;
- cryptographic material;
- datasets;
- hidden state;
- intentionally broken components.

The agent does not receive the complete state directly.

It must discover relevant information through interaction.

7. Formal RL Formulation

An environment can be modeled as a partially observable Markov decision process:

$$\mathcal{E} = (S, A, O, T, R, \gamma)$$

where:

- S = environment states
- A = available actions
- O = observations
- T = transition function
- R = reward function
- γ = discount factor

At timestep t :

$$s_t \xrightarrow{a_t} s_{t+1}$$

The agent receives:

$$o_t = O(s_t)$$

and reward:

$$r_t = R(s_t, a_t, s_{t+1})$$

The objective is:

$$\max_{\pi} E_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

Because the complete environment state is not directly observable, agents must construct internal representations from their observations and interaction history.

8. Terminal Actions

The initial action space may include:

```
EXECUTE_COMMAND
READ_FILE
WRITE_FILE
EDIT_FILE
RUN_TEST
INSPECT_PROCESS
INSPECT_NETWORK
INSTALL_DEPENDENCY
SEND_MESSAGE
ASK_AGENT
DELEGATE
REVIEW
```

RETRY
STOP

The actual environment should expose a constrained action interface rather than unrestricted host access.

9. Multi-Agent Model

Agent Arena supports multiple autonomous agents operating over the same underlying environment.

Each agent may have:

- a distinct role;
- distinct permissions;
- distinct observations;
- separate memory;
- shared memory;
- separate objectives;
- communication capabilities.

A multi-agent configuration may be represented as:

$$\mathcal{M} = \{A_1, A_2, \dots, A_n\}$$

Each agent has a policy:

$$\pi_i(a_i | o_i, h_i)$$

where:

- o_i is the current observation;
 - h_i is the agent's history;
 - a_i is the selected action.
-

10. Agent Roles

The initial implementation should support role specialization.

Explorer

Responsible for environmental discovery.

Typical actions:


```
ls
find
grep
cat
ps
netstat
inspect logs
inspect configuration
```

Primary objective:

Maximize useful information gained from the environment.

Researcher

Responsible for technical reasoning.

Examples:

- derive mathematical approaches;
- analyze algorithms;
- investigate dependencies;
- formulate hypotheses;
- interpret evidence.

The researcher may optionally have restricted terminal access.

Executor

Responsible for implementation and experimentation.

Examples:

- write scripts;
 - modify source code;
 - execute experiments;
 - run tests;
 - install dependencies.
-

Critic

Responsible for challenging proposed solutions.

The critic should actively search for:

- incorrect assumptions;
 - incomplete fixes;
 - hidden regressions;
 - invalid reasoning;
 - benchmark exploitation.
-

Verifier

Responsible for independent validation.

The verifier should ideally have a different information path from the executor.

The objective is:

Determine whether the proposed solution actually satisfies the task.

Orchestrator

Responsible for allocating actions among agents.

Potential decisions:

ACT
DELEGATE
ASK
REVIEW
RETRY
STOP

The orchestrator is the primary candidate for RL optimization.

11. Partial Observability

Agents should not necessarily receive identical observations.

Example:

```
Explorer
  ↓
Files + Logs

Researcher
  ↓
Task + Explorer Report

Executor
  ↓
Task + Researcher Report

Verifier
  ↓
Proposed Solution
```

This forces agents to communicate useful information.

It also prevents the benchmark from becoming a simple collection of independent model calls.

12. Communication Model

Communication should be treated as an explicit action.

Example:

```
{
  "sender": "researcher",
  "recipient": "executor",
  "message":
    "The signatures exhibit approximately 12 bits of nonce leakage. Use the lattice
    formulation described in the attached analysis."
}
```

Communication has a cost.

A useful reward model therefore includes:

$$R = R_{task} - \lambda_c C_{communication}$$

This allows experiments into communication efficiency.

13. Shared Memory

The system should provide structured shared memory.

Possible categories:

```
FACT
HYPOTHESIS
EXPERIMENT
RESULT
WARNING
DECISION
ARTIFACT
```

Example:

```
{
  "type": "FACT",
  "author": "explorer",
  "content": "signatures.json contains 128 ECDSA signatures",
  "confidence": 0.98
}
```

Shared memory should preserve provenance.

Agents should be able to determine:

- who produced a fact;
- when it was produced;
- what evidence supports it;
- whether another agent disputed it.

14. Reward Architecture

A central design principle is:

Do not use only final task success as the reward.

A composite reward can be defined as:

$$R = w_s R_{success} + w_p R_{progress} + w_i R_{information} + w_r R_{recovery} - w_c C_{cost} - w_d C_{destructive} - w_w C_{waste}$$

Where:

Success

Did the agent solve the task?

Progress

Did the agent achieve a meaningful intermediate milestone?

Information gain

Did the agent discover information relevant to the task?

Recovery

Did the agent correctly recover after an unsuccessful approach?

Cost

How many tokens, commands, compute resources, and seconds were consumed?

Destructive behavior

Did the agent perform unsafe or unnecessary modifications?

Waste

Did the agent repeatedly perform actions that provided little information?

15. Example Reward Table

Event	Reward
Relevant file discovered	+0.05
Correct hypothesis	+0.10
Useful experiment	+0.05
Correct intermediate result	+0.20
Successful verification	+0.20
Final solution	+1.00

Event	Reward
Repeated failed action	-0.03
Unnecessary command	-0.01
Destructive modification	-0.20
Timeout	-0.50

These values are initial research parameters, not fixed requirements.

Reward shaping must be experimentally validated because poorly designed intermediate rewards can cause agents to optimize the metric rather than the underlying objective.

16. Reward Hacking

Reward hacking is a first-class research concern.

An agent should not be able to receive high reward simply by:

- generating large quantities of logs;
- repeatedly executing cheap commands;
- producing plausible explanations;
- modifying evaluation infrastructure;
- exploiting verifier weaknesses;
- manipulating metrics.

Therefore:

The evaluator must be isolated from the agent whenever possible.

Ground truth, evaluation logic, and protected artifacts should not be accessible to the agent.

17. Evaluation

Evaluation should have multiple dimensions.

Correctness

Did the agent achieve the actual objective?

Efficiency

How much computation was required?

Reliability

How consistently does the agent solve the task?

Recovery

How effectively does the agent recover from failure?

Safety

Did the agent make unnecessary or dangerous changes?

Collaboration

Did multiple agents produce measurable benefit?

Communication Efficiency

How much useful work resulted from communication?

18. Multi-Agent Efficiency Metric

Define:

$$G = P_{multi} - P_{single}$$

where P is performance.

Then define collaboration efficiency:

$$E = \frac{P_{multi}}{C_{multi}}$$

where C is total computational cost.

This prevents the benchmark from treating:

3 agents at 3× cost

as automatically superior to:

1 agent

19. Experimental Conditions

The platform should support controlled experiments.

Baseline A

Single generalist agent.

Baseline B

Multiple identical agents.

Baseline C

Specialized agents.

Baseline D

Specialized agents + static orchestrator.

Baseline E

Specialized agents + learned orchestrator.

Baseline F

Learned orchestrator + adaptive communication.

The same task seeds and environment configuration should be used across conditions.

20. Curriculum Learning

Tasks should have measurable difficulty.

Example:

```
Level 1  
Simple deterministic debugging  
  
Level 2  
Multi-file debugging  
  
Level 3  
Ambiguous failure  
  
Level 4  
Long-horizon debugging  
  
Level 5  
Multi-service incident  
  
Level 6  
Conflicting evidence  
  
Level 7  
Partial observability  
  
Level 8  
Multi-agent coordination
```

Difficulty can be adapted automatically.

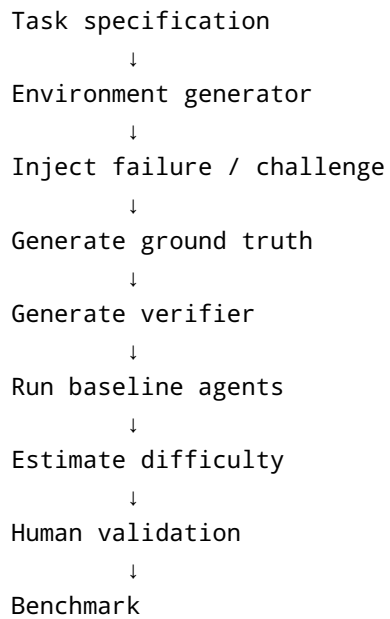
For example:

```
if success_rate > 0.80:  
    increase_difficulty()  
  
elif success_rate < 0.30:  
    decrease_difficulty()
```

The objective is to create a training curriculum rather than a static benchmark.

21. Task Generation

A long-term objective is automated environment generation.



Generated environments must be independently validated.

Automatic task generation should never automatically imply benchmark validity.

22. Initial Environment: Cryptographic Key Recovery

The existing cryptography repository provides the first environment.

The challenge involves ECDSA signatures with biased nonces and requires recovery of the private key through an appropriate cryptanalytic approach.

This environment is valuable because it requires:

- repository exploration;
- mathematical reasoning;
- cryptographic knowledge;
- data inspection;
- code execution;
- dependency management;
- computational experimentation;
- final cryptographic verification.

It therefore provides a strong initial test of long-horizon agent behavior.

23. Future Environment Categories

Agent Arena should expand beyond cryptography.

Security

- log investigation;
- vulnerability analysis;
- compromised host investigation;
- suspicious process analysis.

Linux / Systems

- broken services;
- resource exhaustion;
- permission failures;
- filesystem corruption.

Networking

- service connectivity;
- DNS failures;
- TLS configuration;
- routing problems.

DevOps

- failed deployment;
- broken CI pipeline;
- container failures;
- Kubernetes incidents.

Databases

- inconsistent records;
- failed migrations;
- corrupted indexes;
- transaction failures.

Software Engineering

- unfamiliar repository;
- multi-file bug;
- performance regression;
- dependency conflict.

Distributed Systems

- race conditions;
- partial failures;
- inconsistent state;
- message delivery problems.

Forensics

- reconstructing events from logs;
 - identifying root cause;
 - determining attack paths.
-

24. Trajectory Data

Every execution should produce a structured trajectory.

Example:

```
{
  "task_id": "ecdsa_nonce_bias_001",
  "episode_id": "episode_00042",
  "agent_id": "executor_01",
  "timestamp": "...",
  "observation": "...",
  "action": {
    "type": "EXECUTE_COMMAND",
    "command": "python solve.py"
  },
  "result": {
    "exit_code": 1,
    "stdout": "...",
    "stderr": "..."
  },
  "reward": -0.03,
  "environment_state_hash": "..."
}
```

A complete episode should include:

- actions;
- observations;
- tool calls;

- messages;
 - filesystem changes;
 - tests;
 - errors;
 - rewards;
 - timing;
 - resource consumption;
 - final state.
-

25. Dataset Structure

A dataset could eventually look like:

```
datasets/  
├─ tasks/  
│   ├── task_0001.json  
│   ├── task_0002.json  
│   └── ...  
├─ environments/  
│   ├── task_0001/  
│   ├── task_0002/  
│   └── ...  
├─ trajectories/  
│   ├── agent_a/  
│   ├── agent_b/  
│   └── ...  
└─ evaluations/  
    ├── results.jsonl  
    └── aggregate.json
```

26. Task Specification

A task should contain:

```
id: ecdsa_nonce_bias_001  
  
name: ECDSA Nonce Bias Recovery
```

```
category: cryptography

difficulty: expert

objective:
  type: state_change
  description: Recover and verify the private key.

environment:
  runtime: docker
  network: isolated
  cpu_limit: 4
  memory_limit: 8gb
  timeout: 1200

constraints:
  allow_network: false
  protected_paths:
    - /evaluation
    - /ground_truth

evaluation:
  correctness: true
  efficiency: true
  safety: true
  trajectory: true

agents:
  max_agents: 5
  communication: enabled
  partial_observability: true
```

27. Environment Isolation

Security is critical because agents will execute arbitrary commands.

The execution environment should use:

- isolated Docker containers;
- resource limits;
- controlled networking;
- read-only evaluator assets;
- separate ground-truth storage;

- filesystem snapshots;
- execution timeouts;
- process limits.

Benchmark infrastructure should prioritize reproducibility and isolation. This is consistent with current benchmark infrastructure practices such as OpenHands' use of isolated Docker workspaces and pinned evaluation dependencies.

28. Reproducibility

Every experiment should record:

```
Task version
Environment image
Container digest
Agent version
Model version
Model configuration
Prompt version
Random seed
Dependency versions
Resource limits
Evaluation version
```

A result should be reproducible from a single experiment manifest.

29. Research Experiments

The platform should support systematic experiments rather than anecdotal demonstrations.

Experiment 1 — Single vs Multi-Agent

Question:

Does specialization improve task success?

Variables:

```
1 generalist
2 specialists
```

3 specialists
5 specialists

Experiment 2 — Communication

Question:

How much communication is optimal?

Variables:

No communication
Free communication
Budgeted communication
Learned communication policy

Experiment 3 — Orchestration

Question:

Can learned orchestration outperform static workflows?

Compare:

Round-robin
Fixed pipeline
LLM manager
RL manager

Experiment 4 — Partial Observability

Question:

Does restricted information improve or degrade collaboration?

Compare:

Full observability
Role-based observability
Strong partial observability

Experiment 5 — Recovery

Question:

Which architecture recovers most effectively from failed attempts?

Inject controlled failures and measure:

Recovery probability
Recovery time
Additional cost
Final success

Experiment 6 — Curriculum Learning

Question:

Does adaptive difficulty improve learning efficiency?

Compare:

Random tasks
Static curriculum
Adaptive curriculum

30. RL Strategy

The first implementation should **not** attempt to train the underlying foundation model from scratch.

Instead, focus on learning the orchestration policy.

The LLM agents remain relatively fixed.

The RL component learns:

```
state
↓
select agent
↓
select action
↓
observe result
↓
update policy
```

This dramatically reduces the initial computational requirements.

Potential approaches can later include:

- policy optimization;
- offline RL;
- preference optimization;
- trajectory ranking;
- reward-weighted training;
- hierarchical RL;
- multi-agent RL.

The appropriate algorithm should be selected experimentally rather than assumed in advance.

31. Offline RL

Trajectory logs can become a training dataset.

For example:

```
Observation
Action
Outcome
Reward
Next Observation
```

This allows learning from previously collected agent behavior before introducing online learning.

A useful development sequence is:

```
Rule-based baseline
↓
Collect trajectories
↓
Analyze successful trajectories
↓
Train offline policy
↓
Evaluate
↓
Online RL
```

This is preferable to immediately attempting expensive online training.

32. Multi-Agent RL Formulation

For cooperative agents:

$$\pi = (\pi_1, \pi_2, \dots, \pi_n)$$

with shared objective:

$$J = E \left[\sum_t \gamma^t R_t \right]$$

For mixed incentives:

$$R_i = \alpha R_{team} + (1 - \alpha) R_i$$

where α controls the degree of cooperation.

This creates an experimental parameter for studying team incentives.

Research on LLM-based multi-agent RL highlights coordination and communication as fundamental challenges that differ from single-agent RL.

33. Anti-Cheating Architecture

The benchmark must explicitly defend against:

- reading hidden ground truth;
- modifying the evaluator;

- modifying tests;
- exploiting verifier bugs;
- accessing host resources;
- network-based answer retrieval when prohibited;
- hardcoding expected outputs;
- manipulating reward instrumentation.

The agent should operate in a clean environment while the evaluator operates separately.

34. Benchmark Validity

A task should not enter the official benchmark merely because an LLM generated it.

Each task should undergo:

1. Automated validation
2. Reference solution validation
3. Independent evaluator validation
4. Baseline-agent testing
5. Human review
6. Difficulty calibration
7. Leakage review

This is particularly important as automated task-generation pipelines become increasingly common in agent training research. SWE-rebench demonstrates the value of large-scale executable environments, but also emphasizes filtering and validation of generated instances.

35. Repository Architecture

The proposed implementation structure is:

```
agent-arena/
|
├── README.md
├── LICENSE
├── pyproject.toml
|
├── arena/
|   ├── environments/
|   |   ├── base.py
|   |   ├── docker.py
|   |   └── snapshots.py
```

```
|
|
|— agents/
|   |— base.py
|   |— explorer.py
|   |— researcher.py
|   |— executor.py
|   |— critic.py
|   └─ verifier.py
|
|— orchestration/
|   |— base.py
|   |— sequential.py
|   |— hierarchical.py
|   └─ learned.py
|
|— communication/
|   |— messages.py
|   └─ memory.py
|
|— evaluation/
|   |— evaluator.py
|   |— rewards.py
|   |— metrics.py
|   └─ safety.py
|
|— rl/
|   |— trajectories.py
|   |— replay.py
|   |— policies.py
|   └─ training.py
|
|— runner/
|   |— episode.py
|   └─ experiment.py
|
|— tasks/
|   |— cryptography/
|   |— security/
|   |— systems/
|   |— networking/
|   |— devops/
|   └─ debugging/
|
|— experiments/
|   |— single_agent/
|   └─ multi_agent/
```

```
| | | communication/
| | | └─ orchestration/
| |
| | └─ datasets/
| | | └─ trajectories/
| | | └─ evaluations/
| |
| | └─ scripts/
| |
| └─ docs/
| | └─ architecture.md
| | └─ environments.md
| | └─ agents.md
| | └─ reinforcement_learning.md
| | └─ evaluation.md
| | └─ experiments.md
```

36. Development Roadmap

Phase 0 — Formalize Existing Work

Convert the current cryptography repository into a standardized task.

Deliverables:

- task specification;
- Docker environment;
- evaluator;
- reference solution;
- task metadata;
- trajectory schema.

Phase 1 — Single-Agent Arena

Implement:

```
Environment
Agent
Runner
```

Evaluator
Trajectory logger

Do not introduce RL yet.

Goal:

Establish a reliable experimental baseline.

Phase 2 — Multi-Agent Arena

Implement:

Explorer
Researcher
Executor
Critic
Verifier

Add:

- communication;
 - shared memory;
 - role-specific observations.
-

Phase 3 — Evaluation Framework

Implement metrics for:

success
efficiency
cost
recovery
safety
communication
collaboration

Phase 4 — Multiple Environments

Create approximately:

```
5 cryptography
5 debugging
5 systems
5 security
```

The goal is diversity, not volume.

Phase 5 — Orchestration

Implement:

```
Round-robin
Static workflow
LLM manager
Learned manager
```

Phase 6 — RL

Begin with offline trajectory learning.

Then introduce online learning.

Phase 7 — Curriculum

Implement:

- task difficulty estimation;
 - adaptive task selection;
 - curriculum generation.
-

Phase 8 — Benchmark Release

Publish:

- task dataset;
 - evaluator;
 - environment images;
 - baseline agents;
 - experiment configurations;
 - benchmark leaderboard;
 - research documentation.
-

37. MVP Definition

The first serious milestone should be deliberately small.

The MVP should contain:

```
1 environment
5 agent roles
1 multi-agent workflow
1 single-agent baseline
1 evaluator
1 trajectory format
1 reward function
5 experiments
```

The initial cryptography task should be the environment.

The first research comparison should be:

```
Single Agent
      vs
Specialized Multi-Agent Team
```

Then:

```
Static Team
      vs
Learned Orchestrator
```

Do not build automated task generation, large-scale RL, cloud execution, and a leaderboard simultaneously.

The first objective is to establish that the experimental methodology works.

38. Success Criteria

The project should be considered successful if it can demonstrate at least one of the following:

Research result

A multi-agent architecture consistently outperforms a single-agent baseline on a meaningful class of tasks.

RL result

A learned orchestration policy improves task success or reduces cost relative to a fixed policy.

Behavioral result

Trajectory analysis identifies meaningful differences between agent architectures that binary success metrics cannot capture.

Benchmark result

The environment suite exposes meaningful capability gaps between current autonomous agents.

Infrastructure result

Researchers can reproduce experiments using standardized task/environment specifications.

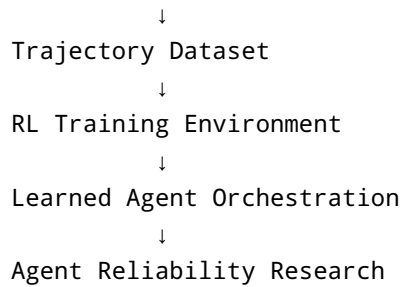
39. Long-Term Vision

The long-term goal is not simply to create another benchmark.

Agent Arena should become an experimental infrastructure layer for autonomous agents.

The intended progression is:

```
Difficult Terminal Tasks
      ↓
Interactive Benchmark
      ↓
Multi-Agent Environment
```



The final system should allow a researcher to ask:

"Given a task and a set of autonomous agents, what organizational structure, communication strategy, and decision policy produces the most reliable solution at the lowest cost?"

That is the central research problem.

40. Positioning

Agent Arena should not position itself as simply another coding benchmark.

Existing infrastructure already covers substantial parts of the terminal-agent evaluation problem. TUA-Bench, for example, evaluates general-purpose terminal-use agents across 120 execution-based tasks, while OpenHands maintains standardized benchmark infrastructure and isolated execution workspaces.

The differentiation should instead be:

Learning how autonomous agents should organize themselves to solve difficult interactive environments.

The benchmark is the experimental substrate.

The research contribution is the study of:

```
graph TD; A[Agent] --> B[+ Environment]; B --> C[+ Communication]; C --> D[+ Coordination]; D --> E[+ Reward];
```

A list of components for research study, each preceded by a plus sign: Agent, Environment, Communication, Coordination, and Reward.

41. Proposed Research Thesis

A concise thesis for the project is:

Agent Arena investigates whether reinforcement learning can learn efficient coordination and orchestration strategies for specialized language-model agents operating in realistic, partially observable terminal environments.

The project evaluates agents not only by whether they eventually solve a task, but also by how they explore, communicate, recover from failure, consume resources, and maintain safety.

42. Immediate Next Steps

The recommended implementation order is:

1. Standardize the existing cryptography task
- ↓
2. Define Environment API
- ↓
3. Define Agent API
- ↓
4. Define trajectory schema
- ↓
5. Build single-agent runner
- ↓
6. Build evaluator
- ↓
7. Add Explorer / Researcher / Executor
- ↓
8. Add Critic / Verifier
- ↓
9. Add shared memory
- ↓
10. Add communication costs
- ↓
11. Establish single-agent baseline
- ↓
12. Establish multi-agent baseline

-
- ```
graph TD; A[13. Implement orchestrator] --> B[14. Collect trajectories]; B --> C[15. Analyze failure modes]; C --> D[16. Introduce RL];
```
13. Implement orchestrator
  14. Collect trajectories
  15. Analyze failure modes
  16. Introduce RL

The critical principle is:

**Build the measurement system before building the learning system.**

If the environment, evaluator, reward function, and trajectory instrumentation are unreliable, RL will optimize an unreliable signal.

The benchmark and evaluation infrastructure should therefore be treated as the foundation of the project, with RL introduced only after the baseline experimental loop is trustworthy.